

Brand Card Browser

A paginated product configurator built with React, Express, and PostgreSQL — implementing cascading dimension dropdowns that resolve to a unique SKU and price.

952 SKUs
in database

52 Brands
paginated

PERN Stack

MVC Architecture

Vercel · Neon
deployed

1 Project Overview

This project is a take-home engineering assessment requiring the construction of a full-stack product browsing interface backed by a pre-supplied PostgreSQL dataset. The application allows users to browse brands, configure dimension selections, and resolve a unique SKU with its exact price — all within a responsive, paginated UI.

● React 19 · TypeScript

● Node.js · Express

● PostgreSQL · Raw SQL

● Vite · Tailwind CSS

● Vercel · Neon

Business Problem

A product catalog with 948 SKUs across 52 brands, where each product is identified by a unique *brand + length + width* combination. Users need a frictionless way to configure and identify a specific product SKU and its price.

Solution

A paginated brand card grid where each card presents cascading dropdowns — length drives the available width options, and both together resolve a guaranteed unique SKU and price from the database.

Objectives

Implement clean MVC architecture, raw SQL without an ORM, a properly decoupled frontend, and ship to a publicly accessible production URL — all in one focused development session.

Dataset

Single products table: sku, brand, length, width, price. 52 brands, dimensions 30–50, prices \$10–\$100. Unique constraint on (brand, length, width).

2 Requirement Analysis

Before writing a single line of implementation code, the specification was systematically decomposed into explicit requirements and implied engineering expectations — a distinction that shaped multiple design decisions.

Explicit Requirements

- Query products table, group by brand — each card represents one unique brand
- Default card state shows brand name and price range ($\min(\text{price}) - \max(\text{price})$)
- Length dropdown shows all distinct lengths for that brand, sorted ascending
- Width dropdown filtered to only widths valid for the selected length, for that specific brand
- Width dropdown disabled until a length is chosen

- When both dimensions selected: show the exact matching SKU and price
- Display 12 cards per page with previous, next, and numbered pagination controls

Implied Requirements (Identified During Analysis)

- Width selection must reset whenever length changes — stale selections would produce incorrect SKU lookups
- Dropdown values must be sorted numerically ascending (not lexicographically — "30, 31..." not "30, 40, 50, 31...")
- Prices displayed as \$XX integer prefix — not raw 36.00 decimal strings
- Visible loading state while card options fetch on mount (12 concurrent requests per page)
- Graceful error state if any API call fails — must not crash the card or page
- Race condition guard on SKU fetch — rapid length changes can produce stale responses
- Pagination shows total page count, not just previous/next arrows

Edge Cases Explicitly Documented

Length Change Reset

Changing the length dropdown must clear the selected width and any resolved SKU, since the previously selected width may not be valid for the new length.

Width Pre-filtering

The database guarantees that any (brand, length, width) triple maps to exactly one SKU — the cascade logic must honor this constraint without allowing invalid combinations.

3 Planning & Technical Decisions

Every technology choice was deliberate and justified against the problem constraints — not selected from habit or convenience.

React 19 + TypeScript

Frontend

React's component model maps naturally to the card-per-brand structure. TypeScript strict mode ensures the API contract between frontend and backend is enforced at compile time — catching mismatches before they reach the browser. The custom hook pattern (useBrands, useBrandCard) cleanly separates data-fetching concern from rendering, keeping components thin and testable.

Vite + Tailwind CSS

Tooling

Vite's near-instant HMR eliminated rebuild wait-time during iterative UI development. Tailwind CSS provides utility-first styling without a CSS abstraction layer — the constraint of working with preset classes enforces visual consistency without needing a design system.

Express + Node.js

Backend

Lightweight and explicit — Express adds no magic over raw Node.js HTTP while providing clean middleware composition for CORS, JSON parsing, and centralized error handling. For a read-only API serving three endpoints, this is the right weight of tool.

Raw pg SQL

No ORM

The assessment spec implied raw SQL was expected. Beyond compliance, raw SQL offers complete control over query shape — the COUNT(*) OVER() window function for single-query pagination and the numeric-aware brand sort (REGEXP_REPLACE → CAST → INTEGER) would be difficult or impossible to express through ORM abstractions without dropping down to raw strings anyway.

MVC Architecture

Backend Structure

A strict four-layer separation (Route → Controller → Service → Repository) was chosen to make the codebase legible to any reviewer. SQL lives exclusively in the repository layer. Business logic (computing totalPages, building the widthsByLength map) lives in the service layer. The controller only handles HTTP parsing and response serialization. This is intentional over-structure for a three-endpoint API — it demonstrates architectural discipline.

Neon PostgreSQL **Hosting**

Neon provides a serverless PostgreSQL instance on a free tier with a connection string compatible with standard `pg.Pool`. It allowed the same SQL dump used in local development to be imported directly into production — zero schema drift between environments. SSL was enabled with `rejectUnauthorized: false` to work with Neon's certificate authority.

Options Prefetch Strategy **Performance**

Each `BrandCard` fetches its complete `widthsByLength` map on mount via `/options`, rather than making a new request each time the user selects a length. This trades one slightly larger response for zero latency on the length→width cascade — width filtering happens entirely client-side after that single fetch. 12 cards mount concurrently per page, so all 12 options requests fire in parallel.

4 Project Architecture

The project is organized as a monorepo with two independent applications sharing no runtime dependencies — a Vite React client and an Express API server — connected only through a well-defined HTTP API contract.

```
Interview Assessment/
├─ client/                               # Vite React app (port 5173)
│   └─ src/
│       ├─ components/                  # Reusable UI components
│       │   └─ BrandCard/               # Card with cascading dropdowns
│       │   └─ Pagination/             # Page navigation controls
│       │       └─ ui/                  # Generic primitives (Dropdown, Spinner, Error)
│       ├─ hooks/                      # useBrands, useBrandCard
│       ├─ pages/                      # HomePage (route-level)
│       ├─ services/                   # brandService.ts – all fetch calls
│       ├─ types/                      # TypeScript API response interfaces
│       └─ utils/                      # formatPrice, formatPriceRange
└─ server/                             # Express API (port 8001)
    └─ src/
        ├─ server.ts                   # Entry point – dotenv + conditional listen
        ├─ app.ts                      # Express setup, middleware, routes
        ├─ errors.ts                   # AppError class (typed HTTP errors)
        ├─ controllers/                # HTTP layer – parse, validate, respond
        ├─ services/                   # Business logic, data transformation
        ├─ repositories/               # SQL queries only
        ├─ routes/                     # URL → controller binding
        ├─ db/                         # pg Pool singleton
        ├─ middleware/                 # Global error handler
        └─ types/                      # Shared TypeScript interfaces
```

Backend MVC Request Flow

```
API REQUEST LIFECYCLE
GET /api/brands?page=1

— Routing —
brandRoutes → GET /brands → getBrands()

— Controller —
Parse query params: page, limit
Apply defaults (page=1, limit=12) + clamp bounds
Call brandService.getBrands(page, limit)

— Service —
Calculate offset = (page - 1) * limit
Call brandRepository.findBrandsPaginated(limit, offset)
Compute totalPages = Math.ceil(total / limit)
Return { brands, total, page, totalPages }

— Repository —
SELECT brand, MIN(price), MAX(price),
        COUNT(*) OVER() AS total_count
```

```
FROM products
GROUP BY brand
ORDER BY numeric-sort, brand ASC
LIMIT $1 OFFSET $2
```

— *Response* —

```
res.json({ data: { brands, total, page, totalPages } })
```

Frontend Data Flow

```
CLIENT STATE FLOW

HomePage mounts
└─ useBrands(page) → GET /api/brands?page=N
    └─ renders 12 × BrandCard + Pagination

BrandCard mounts (×12 concurrently)
└─ useEffect → GET /api/brands/:brand/options
    └─ stores { lengths, widthsByLength } locally

User selects Length
└─ resets selectedWidth + skuResult           ← no API call
└─ computes availableWidths from widthsByLength[length]

User selects Width
└─ GET /api/brands/:brand/sku?length=X&width=Y
    └─ displays SKU + exact price

Page change
└─ useBrands refires → all BrandCard state resets naturally
```

API Contract

GET /api/brands

Query: page, limit
Returns paginated brand list with price range per brand

GET /api/brands/:brand/options

Returns all valid lengths[] and a widthsByLength map for client-side cascade filtering

GET /api/brands/:brand/sku

Query: length, width
Returns exact sku and price or 404

5 Development Phases

Development followed a deliberate phase-by-phase progression from infrastructure outward — database first, then API, then UI — ensuring each layer was verified before building on top of it.

Phase 0 Project Initialization

- Goal:** Establish full project skeleton before writing any logic
- Initialized monorepo: server/ (Express + TypeScript MVC) and client/ (Vite + React + Tailwind)
 - Configured strict TypeScript across both apps — no any policy enforced from day one
 - Wrote all server skeleton files with typed stub implementations returning {"data":{}}
 - Defined shared TypeScript interfaces for Brand, PaginatedBrands, BrandOptions, SkuResult
 - Created documentation: CLAUDE.md, project-spec.md, docs/architecture.md
 - Set up .gitignore and .env.example — secrets excluded from git from the first commit

Phase 1 Database Setup

- Goal:** Establish verified database connection before any SQL is written
- Installed PostgreSQL 15 locally on port 5432 and created the brand_cards database
 - Restored the provided SQL dump — verified 948 SKUs across 52 brands loaded correctly
 - Configured pg.Pool singleton in server/src/db/pool.ts reading from DATABASE_URL environment variable

- Ran a test query via `ts-node` to confirm pool connection and data integrity before proceeding
- Key insight: confirmed price is stored as `numeric(18,2)` — the `pg` driver returns this as a string, requiring explicit `parseFloat()` conversion in the service layer

● **Phase 2 Backend Foundation**

- Goal:** Validate full MVC wiring before filling in SQL logic
- Completed MVC scaffold: controller → service → repository → routes connected end-to-end
 - Added `try/catch + next(err)` to all controller stubs — error handler wired before logic
 - Implemented `AppError` class for typed HTTP errors with status codes
 - Confirmed all three route stubs returning `{"data": {}}` on a live server before any real logic
 - Zero TypeScript compilation errors — strict mode maintained throughout

● **Phase 3 API Implementation**

- Goal:** Implement all three endpoints with production-quality SQL
- Implemented `findBrandsPaginated` using `COUNT(*) OVER()` window function — single query for both data and total count
 - Implemented numeric brand sort via `REGEXP_REPLACE → NULLIF → CAST AS INTEGER` — brands like "Brand 10" sort after "Brand 9"
 - Implemented `findBrandLengthWidthPairs` returning all valid dimension combinations ordered ascending
 - Built `widthsByLength` map in the service layer — client-side cascade computed from one response, not per-interaction API calls
 - Added `UNIQUE (brand, length, width)` constraint to database — guarantees SKU lookup always returns exactly one result
 - Added `idx_products_brand` B-tree index — eliminates full-table scan on brand-filtered queries
 - Verified: 52 brands, 5 pages, correct cascade options, prices formatted correctly

● **Phase 4 Frontend Foundation**

- Goal:** Scaffold all client types, services, hooks, and component shells
- Verified TypeScript interfaces align exactly with API response shapes — type safety across the HTTP boundary
 - Implemented `useBrands` hook with cancellation pattern — prevents `setState` on unmounted components during page transitions
 - Implemented `brandService.ts` with all three `fetch` calls and `encodeURIComponent` on brand param
 - Implemented `Dropdown` ui component: controlled select, disabled state, custom SVG chevron arrow
 - Confirmed production build clean: 24 modules, 6.62 kB Tailwind CSS bundle, zero TypeScript warnings

● **Phase 5 UI Implementation**

- Goal:** Implement full `BrandCard` and `Pagination`, wire cascading dropdown logic
- Implemented `useBrandCard` hook: options fetch on mount, cascade state machine, race condition guard via `useRef` generation counter
 - Race condition: rapid length changes can fire multiple SKU requests — only the latest response is applied, stale responses silently discarded
 - Price display logic: shows price range until both dropdowns selected, replaced by `$price · SKU` on resolve
 - Implemented `Pagination`: numbered buttons, `← →` controls, disabled at boundaries, returns null for single-page results
 - Applied polish: sticky header with blue accent, card hover shadow, refined pagination, accessible disabled states

•

Phase 6

Integration & Code Audit

Goal: End-to-end integration testing and code quality audit

- Identified and fixed critical bug: `dotenv` must be imported in `server.ts` (the true entry point) before `app.ts` — importing it in `app.ts` meant the Pool initialized before `DATABASE_URL` was loaded
- Conducted full code audit: validation coverage, SQL safety, TypeScript strictness, error propagation
- Added input validation hardening: brand param max 100 chars, length/width must be positive integers
- Confirmed cascading dropdown behavior across all 52 brands — no edge cases surfaced

•

Phase 7

Deployment

Goal: Ship both frontend and backend to production

- Migrated PostgreSQL from local to Neon hosted — same SQL dump, SSL enabled in pool config
- Configured `server/vercel.json` for serverless deployment — all routes to `src/server.ts`
- Added conditional `app.listen()` in `server.ts` — skips local server start when running as Vercel serverless function
- Deployed client to Vercel — configured `VITE_API_URL` environment variable pointing to production API
- Set `CORS_ORIGIN` on server to client's Vercel domain — locked down CORS to production origin

6

Engineering Workflow

1

Requirement Analysis & Edge Case Discovery

Before any implementation, the PDF specification was analyzed collaboratively to surface implied requirements — the width-reset-on-length-change behavior, the race condition risk on rapid selections, and the numeric sort requirement on brand names were all identified during this planning phase rather than discovered during debugging.

2

Architecture Design & Trade-off Discussion

The key architectural decision — fetching the full `widthsByLength` map once on card mount versus making an API call per length selection — was explored and validated before writing the `/options` endpoint. The trade-off (one slightly larger response vs. zero latency cascade) was explicitly reasoned through.

3

Phased Implementation Strategy

The eight-phase sequence was planned before writing code. Each phase had defined goals and verification criteria — a phase was not considered complete until its outputs were confirmed working. This prevented the common failure mode of building on unverified foundations.

4

SQL Query Design

The `COUNT(*) OVER()` window function approach for single-query pagination was discussed and validated against the alternative (two separate queries). The numeric brand sort regex was developed iteratively — tested against actual brand names in the dataset before being committed.

5

Code Audit & Bug Discovery

After each implementation phase, a structured code review was conducted to surface issues before integration testing. The `dotenv` load-order bug (pool initializing before environment variables were available) was caught during a Phase 6 audit — before it could cause a production failure.

6

Documentation Workflow

`CLAUDE.md` served as a living project brief — conventions, architecture decisions, and API shape defined at the start and kept current throughout. This ensured consistency across the full session and would allow any reviewer to orient quickly. Lightweight docs (`docs/architecture.md`, `project-spec.md`) were written as engineering artifacts, not generated post-hoc.

7

Major Features Implemented



Paginated Brand Cards



Cascading Dropdown Logic

12 cards per page in a responsive 1→2→3 column grid. Each card shows brand name, price range, and dimension dropdowns. Smooth scroll-to-top on page change.

Length selection filters the width options client-side with zero latency. Width selection triggers the SKU fetch. Length change resets width and clears any resolved SKU.



Loading & Error States

Page-level loading spinner during brand fetch. Per-card loading spinners during options fetch and SKU fetch. Inline and page-level error states with clean recovery.



Race Condition Guard

A useRef generation counter in useBrandCard tags each SKU request. Only responses matching the latest ID update state — stale responses from slow or cancelled requests are silently discarded.



Single-Query Pagination

The COUNT(*) OVER() window function returns the total row count alongside each page of data — eliminating a second database round-trip for the count query.



Security & Input Validation

Parameterized SQL queries throughout — no injection risk. Brand param validated: trimmed, max 100 chars. Length/width validated as positive integers. encodeURIComponent on all URL params.



Modular Backend

Four-layer MVC separation: Routes bind URLs to controllers. Controllers validate HTTP input. Services own business logic. Repositories contain only SQL. No cross-layer SQL escaping.



Numeric Brand Sort

Brands are sorted numerically using REGEXP_REPLACE → CAST AS INTEGER — ensuring "Brand 9" appears before "Brand 10" rather than falling into lexicographic ordering.

8 Technical Challenges & Solutions

PROBLEM Race condition on rapid length selection

Challenge: A user who changes the length dropdown quickly fires multiple concurrent SKU requests. The slower earlier request could resolve after the newer one, overwriting correct state with stale data.

SOLUTION

A useRef(0) counter (skuRequestId) in useBrandCard is incremented on every length change. Each SKU fetch captures its current ID at call time. The response handler checks skuRequestId.current === capturedId before applying any state update — stale responses are silently discarded.

PROBLEM pg returns NUMERIC columns as strings

Challenge: PostgreSQL's numeric(18,2) type is returned by the pg driver as a JavaScript string ("36.00"), not a number. Passing this directly to the frontend would break price formatting and TypeScript types.

SOLUTION

Explicit parseFloat() conversion applied in the service layer for all price fields — minPrice, maxPrice, and SKU price. The conversion lives in the service, not the repository, keeping SQL results raw and conversion logic in business logic where it belongs.

PROBLEM Environment variables not loaded before Pool initialization

Challenge: dotenv was imported in app.ts, but db/pool.ts is imported by app.ts at module load time. This meant the Pool was initialized — reading DATABASE_URL — before dotenv.config() had executed.

SOLUTION

Moved import 'dotenv/config' to server.ts — the true entry point that Node.js loads first. This guarantees environment variables are populated before any module that reads them is initialized. A subtle but production-critical ordering issue.

PROBLEM Lexicographic vs. numeric brand sort

Challenge: Brands named "Brand 1" through "Brand 52" sort incorrectly under standard `ORDER BY brand ASC` — "Brand 10" appears before "Brand 2" because the sort is lexicographic, not numeric.

SOLUTION

```
ORDER BY
  CAST(NULLIF(REGEXP_REPLACE(brand, '^[0-9]', '', 'g'), '') AS INTEGER) NULLS LAST,
  brand ASC
```

Strips non-digit characters from brand name, casts to integer for numeric comparison, falls back to alphabetic sort for brands with no numeric component. `NULLS LAST` ensures non-numeric brands appear after numbered ones.

PROBLEM Vercel serverless: `server.listen()` must not fire

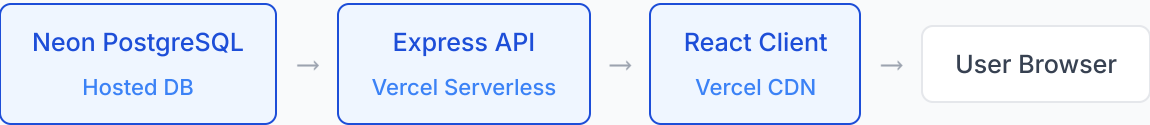
Challenge: Vercel's serverless runtime imports the Express app as a module — calling `app.listen()` inside a serverless function throws an error because there's no persistent process to bind a port to.

SOLUTION

Added a runtime check: `if (!process.env.VERCEL) { app.listen(PORT) }`. The Express app is exported as the default export in all cases. Vercel uses the exported app; local development uses the `listen()` call. One codebase, two deployment targets, no divergence.

9 Deployment

Both applications are deployed to Vercel with a Neon-hosted PostgreSQL instance. The deployment was designed to mirror the local development setup as closely as possible — same SQL schema, same environment variable names, same application code with only runtime checks differentiating local from serverless execution.



Frontend — Vercel

Vite builds the React app to `dist/`. Vercel serves the static bundle from CDN edge nodes. `VITE_API_URL` is set in the Vercel dashboard to the deployed API URL — baked into the bundle at build time.

Backend — Vercel Serverless

`vercel.json` routes all requests to `src/server.ts` via the `@vercel/node` runtime. The Express app is exported as a module. `app.listen()` is skipped when `process.env.VERCEL` is set.

Database — Neon PostgreSQL

SQL dump imported directly to Neon. Connection pool uses `DATABASE_URL` from environment. SSL enabled with `rejectUnauthorized: false` for Neon's certificate chain. Same schema as local development.

Environment Variables

`DATABASE_URL`, `CORS_ORIGIN`, `PORT` on the server. `VITE_API_URL` on the client. No secrets in the repository — all managed through Vercel environment variable dashboard per deployment target.

```
SERVER/VERCEL.JSON
{
  "version": 2,
  "builds": [{ "src": "src/server.ts", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "src/server.ts" }]
}
```

10 Future Improvements

The following improvements are explicitly out of scope for this assessment but represent the natural next investments for a production system.

HIGH PRIORITY Test Coverage Unit tests for service/repository layer. Integration tests hitting a test database. React Testing Library for hook and component tests.	HIGH PRIORITY API Response Caching Brand options are static — a short-lived cache (Redis or in-memory) would eliminate 11 redundant /options requests per page load for the same brands.	HIGH PRIORITY Authentication JWT-based auth layer protecting write endpoints. Role-based access for an admin panel. Session management with refresh token rotation.
MEDIUM Admin Dashboard CRUD interface for adding, editing, and deleting SKUs and brands. Batch import via CSV. Price update workflow with audit trail.	MEDIUM Search & Sorting Global brand search with debounced input. Client-configurable sort order (price, name). URL-persisted filter state via query parameters.	MEDIUM CI/CD Pipeline GitHub Actions: type-check and test on every push. Preview deployments on pull requests. Automated deployment on merge to main.
LOW Containerization Dockerfile for local development parity. Docker Compose to spin up API + PostgreSQL with a single command. Production-ready multi-stage build.	LOW Observability Structured logging (Winston/Pino) with request IDs. Performance monitoring on slow SQL queries. Error tracking with Sentry or similar.	LOW Performance React Query for request deduplication and automatic refetch. Virtualised grid for large datasets. Image optimization for brand assets if added.